

Whole-App Developer Guide & Technical Review

Architecture, Relay data flow, page handbook, data quality, performance, security, and implementation outcomes

Maintained application	apps/bitcraft-local
Review checkpoint	21 August 2026
Implementation reconciliation	22 August 2026
Audience	Junior developers, maintainers, operators, and reviewers

Developer handoff edition

This document preserves dated review evidence while clearly identifying the implemented outcome and remaining deployment or approval gates.

Contents

Contents	2
Executive summary	6
Implemented outcome at 22 August 2026	7
1. Scope, method, and confidence	7
1.1 What was reviewed	7
1.2 Evidence used	8
1.3 Verification snapshot	8
2. Product and domain orientation	8
3. System architecture	9
3.1 Runtime processes	9
3.2 End-to-end shape	10
3.3 Startup flow	10
3.4 Stable and deep modules	10
4. Relay data acquisition and publication	11
4.1 Topology discovery	11
4.2 Relay HTTP path	11
4.3 Typed subscription path	12
4.4 Schema gate, review-time mismatch, and implemented recovery	12
4.5 Normalization	13
4.6 Domain generations and last-good state	13
4.7 Current state versus durable side effects	13
4.8 Health and provenance	14
5. Local API and browser data flow	14
5.1 Provider-neutral game-data route	14
5.2 Specialized APIs	15
5.3 Browser refresh policy	15
5.4 Navigation cache behavior	15

5.5 Unnecessary history request	15
5.6 Startup configuration and auth	16
6. Domain ownership and data completeness	16
6.1 Main domain map	16
6.2 Observed local domain condition	16
6.3 Semantics that upstream data does not currently prove	17
6.4 Additional data-quality investigations	17
7. Frontend shell, navigation, and shared behavior	17
7.1 Entry and route model	17
7.2 Shared chrome and overlays	18
7.3 Access and auth	18
8. Page-by-page developer handbook	19
8.1 Dashboard	19
8.2 Leaderboard	19
8.3 Members	19
8.4 Professions	20
8.5 Craft Monitor	20
8.6 Craft Planning	20
8.7 Inventory	21
8.8 Construction	21
8.9 Research	21
8.10 Local Market	22
8.11 Global Market	22
8.12 Region	23
8.13 Empires	23
8.14 Map	24
8.15 Activity	24
8.16 Public Craft Finder	25
8.17 Craft Calculator	25

8.18 Sync	25
8.19 Admin console	25
8.20 Bot Control	26
8.21 Terms, Privacy, and legal dialogs	27
8.22 Global overlays	28
9. Backend responsibilities beyond Relay	28
10. SQLite ownership, history, and retention	29
10.1 Appropriate durable data	29
10.2 Growth risk	29
10.3 Database operational measurements	30
11. Security, privacy, and delivery review	30
11.1 Positive controls	30
11.2 Rate-limit identity	30
11.3 Discord leasing	31
12. Performance review	31
12.1 Measured local API shape	31
12.2 Hot paths	31
12.3 Recommended performance approach	32
12.4 Candidate budgets after a baseline	32
13. UX, accessibility, and maintainability review	32
13.1 High-value UI fixes	32
13.2 Test posture	33
13.3 AppShell direction	33
14. Junior developer workflow	33
14.1 How to trace a page	33
14.2 How to add a Relay-backed field	33
14.3 How to decide whether to add a table	34
14.4 Verification commands	34
15. Ranked findings and proposed ownership	35

16. Recommended roadmap	35
Implemented locally: restore truth and protect correctness	35
Implemented locally: remove measured waste	36
Remaining after evidence and external gates	36
17. Source map	36
18. Review limitations and next evidence	37
19. Companion documents	37

Review date: 21 August 2026

Maintained application: apps/bitcraft-local

Audience: Junior developers, maintainers, operators, and reviewers

Review type: Whole-application architecture, data-flow, performance, completeness, and maintainability review

Implementation reconciliation — 22 August 2026: Sections that describe observations from the 21 August review retain that date and context. Tasks 1–12 were subsequently implemented on the review branch. Current outcomes and remaining gates are called out explicitly below; dated payload sizes, warning counts, fingerprints, and browser limitations remain historical evidence, not present-tense production claims.

Executive summary

BitCraft Claim Monitor is a local-first settlement operations dashboard for BitCraft. It is not a thin browser client over Relay. Long-running Node processes discover Relay sources, acquire current game data through two distinct mechanisms, normalize that data into provider-neutral domain models, keep durable last-good generations and history in SQLite, and expose only same-origin local APIs to React. The browser then composes those APIs into public operations pages, market and map tools, an administrator console, and a Discord bot console.

The design has several strong foundations:

- Relay wire records are kept behind a provider boundary and do not enter React.
- Typed item and cargo identities and 64-bit identifiers are preserved correctly.
- Current domain data is committed atomically per domain and last-good data survives source outages.
- HTTP acquisition has bounded timeouts, retry, and circuit-breaking.
- The browser refresh controller coalesces work, pauses hidden tabs, retries with backoff, and keeps old data visible during refresh.
- Security-sensitive settings, administrator mutations, CSRF, Discord delivery modes, privacy retention, and deployment identities are deliberately separated.
- The automated suite is broad: the reviewed checkout passed 2,369 tests, with 3 intentional skips and no failures.

The most important findings at the 21 August review checkpoint were:

- 1 **The 21 August local smoke observed global Relay schema drift from the checked-in generated bindings.** The dated observed fingerprint was d8351a1bf26bc73cc9b589b2c7631a9d9042e626e51b8e1bbbc8c1ea74b91ee4, while the manifest then expected 5814c18474097f92cd37c577a9c7f033c820a6d2dd7e679db936ba018a396f8c. The fail-closed gate worked as designed and stopped global catalog reconciliation in that environment.
- 2 **The main browser data route and order-book route had expensive, unbounded public read paths.** The measured dashboard response was about 317 KB before compression, and Global Market Overview could create up to 20 order-book requests per refresh for favorites.
- 3 **Operational history was not bounded.** market_events, market_trades, activity_events, and production contribution history could grow indefinitely, while some health and leaderboard queries scanned broad histories.
- 4 **There was a durability gap between a committed market snapshot and its history/notification side effects.** A crash window existed because the current snapshot committed before an in-memory transition callback, without a durable hand-off on that inspected path.
- 5 **A generic Discord outbox delivery could be claimed by more than one process.** The table had locking fields, but the inspected generic consumer did not atomically lease a row before sending.
- 6 **Multi-domain API responses could contain independently current last-good generations without declaring coherence.** This was an API-contract risk rather than proof of bad displayed values; partial availability was intentional.
- 7 **The browser discarded most per-domain quality metadata.** The server supplied freshness, confidence, age, provenance, and warnings, but React retained mainly a global stale flag.
- 8 **Changing to a page or claim without a cache entry could briefly render the previous scope's data.** Same-scope retained data was desirable during refresh; cross-scope retained data was not.

- 9 **Most pages polled by interval even though design documents described generation-driven invalidation for all open pages.** Only Craft Monitor used the generation event watcher.
- 10 **Every ordinary page requested recent activity history even when it did not render it.** On the smoke instance, this unnecessary request was about 32 KB per refresh cycle.

The recommended sequence is: improve diagnostics, refresh and verify bindings, fix correctness and durability risks, establish route and database baselines, remove the market fan-out and unnecessary refresh work, then introduce retention, richer domain status, and maintainability improvements. The companion implementation plan gives test-first tasks, dependencies, success criteria, and exact source locations.

Implemented outcome at 22 August 2026

Review-time finding	Implemented outcome	Remaining limit or gate
Global schema drift blocked catalog-backed domains	Fresh global/regional schemas were captured; generated output was validated; global and regional typed live verifiers passed; a built smoke returned skills, buildings, research, public crafts, market, and region with no partial errors	Future schema changes still fail closed and require the same capture/generation/deployment process
Cross-scope browser data	Loader state and completions are claim/panel scoped	Interactive throttled browser smoke was unavailable; focused behavior and build evidence passed
Public hot-route cost and market favorite fan-out	Bounded route telemetry and global concurrency gates landed; 20 favorites now use one typed batch request and a bounded generation/scope cache	Per-client profiles remain report-only pending deployed proxy probes and two seven-day observation windows
Unbounded navigation cache and unnecessary history reads	Cache is LRU-bounded to 8 entries, 4 MiB, and 5 minutes; only Dashboard, Activity, and Settlement Market request their owned history projections	Full browser route-loop heap evidence remains open
Market transition crash window	Market generation and compact transition commit atomically; a leased worker dispatcher applies idempotent history/activity/Discord enqueue after restart	No real Discord delivery was tested
Generic Discord duplicate claiming	Rows use renewable SQLite leases and token-conditioned completion	Delivery is at-least-once; the external acknowledgement duplicate window remains
Undeclared mixed generations and discarded quality	meta.coherence, exact dependency generations, and bounded domainStatus warnings/provenance reach React	Coherence describes known local application generations, never simultaneous upstream observation
Craft-only generation watching	All provider-neutral pages use claim/domain-scoped watchers; hidden tabs do not poll; manual/non-provider pages do not create watchers	Named-page browser/telemetry smoke was blocked locally
Startup duplication and popup accessibility	One public bootstrap resolves config/auth/legal/build before claim hooks; popups use the shared Dialog	A separate in-app browser click-through was not claimed; mounted DOM tests passed
Operational-history growth	Disabled/report-only rollups, diagnostics, dry-run, backup checks, and fail-closed readers landed; defaults remain disabled, allowlist empty, and there is no prune route	Production baseline, reader parity, owner/dependency/legal approval, verified production backup/root, seven dry-run days, deployment approval, and enablement remain open

The canonical legal files LICENSE and NOTICE state AGPL-3.0-only. The root package.json still declares MIT. That contradiction was documented but not resolved; contributors must treat the legal files as controlling until a maintainer/legal metadata correction is made.

1. Scope, method, and confidence

1.1 What was reviewed

The review covers the maintained application only:

- React entry point, shell, navigation, pages, dialogs, styles, hooks, and browser data loaders.
- Node HTTP server and worker entry points.
- Relay topology discovery, HTTP cache adapters, typed SpacetimeDB sessions, normalizers, projections, generation repository, and health persistence.

- SQLite current-state, history, user, administrator, Discord, privacy, and operational responsibilities.
- Provider-neutral browser APIs, specialized market and map APIs, administrator APIs, and bot APIs.
- Refresh, notifications, manual refresh, last-good behavior, and errors.
- Tests, build pipeline, deployment topology, asset verification, and runtime smoke behavior.

Historical exports and retired provider code were out of scope unless an active boundary test or migration document referenced them.

1.2 Evidence used

This document combines:

- Targeted source inspection of apps/bitcraft-local and directly related documentation.
- Three independent audits of the frontend, Relay/backend pipeline, and performance/data quality.
- A production-style frontend and server build.
- The complete Node test suite.
- A built smoke server on `http://127.0.0.1:18449`.
- Local API response timing and payload-size sampling.
- Current source-health and domain-envelope inspection.
- Smoke server logs, including the review-time schema mismatch.

This was not a production load test. Timings are local observations from one workstation and should establish relative shape, not production capacity. Browser visual automation could not be completed because the in-app browser dependency failed its trusted-path setup. Build, tests, server health, APIs, payloads, and logs were still verified.

1.3 Verification snapshot

- Build: passed.
- Tests: 2,372 discovered; 2,369 passed; 0 failed; 3 skipped.
- Asset verification: 1,454 assets verified; 9,421 catalog identities; 522 identities use the unavailable/text fallback path.
- Built frontend chunks of note:
- AppShell: 217.57 KB, 67.12 KB gzip.
- AdminPanel: 163.36 KB, 41.08 KB gzip.
- Craft Planning: 94.93 KB, 26.97 KB gzip.
- Map: 70.77 KB, 23.18 KB gzip.
- Market: 59.84 KB, 16.21 KB gzip.
- Local smoke health: healthy HTTP process, version 0.58.0-beta.4, build c081890cc330.

2. Product and domain orientation

The application helps a settlement answer operational questions:

- Who is in the settlement and what are they doing?
- Which crafts are active, who contributed, and what is blocked?
- What materials exist in storage and what construction or research needs them?
- What should the settlement craft next?
- What is listed locally and across regional markets?
- Where are players, claims, resources, roads, terrain, watchtowers, and sieges?

- What happened recently, and which changes need notifications?
- Is the Relay data pipeline, database, Discord integration, and deployment healthy?

The primary business scope is a configured monitored claim and its active regions. The application deliberately rejects cross-claim and out-of-scope region leakage.

Important terms:

- **Claim:** The monitored settlement identity.
- **Domain:** One normalized slice of current data, such as members, crafts, market, or research.
- **Generation:** A validated publication of a domain snapshot.
- **Last-good:** The newest previously valid snapshot retained when a new source read fails.
- **Relay HTTP cache:** Relay's joined HTTP endpoints, used where they already provide proven claim-oriented views.
- **Typed subscription:** An official generated SpacetimeDB client subscription to exact global or regional tables.
- **Projection:** A provider-neutral structure derived from typed rows or Relay HTTP results.
- **Current state:** What the UI should show now.
- **History/event:** A durable observation needed for charts, transitions, notifications, dedupe, or audit.
- **Provider seam:** The interface that hides Relay-specific acquisition from the rest of the application.

3. System architecture

3.1 Runtime processes

The important runtime components are:

1 **React browser application**

- Entry: `apps/bitcraft-local/src/main.tsx`.
- Main coordinator: `apps/bitcraft-local/src/AppShell.tsx`.
- Reads only same-origin `/api/local` routes.
- Never connects to Relay or SpacetimeDB directly.

2 **Node web process**

- Entry: `apps/bitcraft-local/server.mjs`.
- Serves production assets and the provider-neutral API.
- Owns HTTP routing, auth, CSRF, admin operations, specialized projections, and some supervised provider work.

3 **Node worker process**

- Entry: `apps/bitcraft-local/worker.mjs`.
- Starts the same server module in worker role.
- Owns long-running acquisition, collection, reconciliation, history, and delivery responsibilities according to process-role fencing.

4 **SQLite**

- Development path: `apps/bitcraft-local/data/bitcraft-local.sqlite`.
- Production path: `/var/lib/bitcraft-claim-monitor-relay`.
- Stores current last-good envelopes, health, durable events, user/admin state, Discord queues, settings, privacy records, and audit data.

5 **Relay**

- Topology and cache-readiness discovery.

- Joined Relay HTTP cache endpoints.
- Global and regional SpacetimeDB databases consumed through generated bindings.

6 Discord

- Bot gateway and administrative configuration.
- Durable notification and delivery workflows.
- Preview deployments use record mode unless an explicit sandbox test is authorized.

3.2 End-to-end shape



The deepest architectural boundary is `apps/bitcraft-local/src/server/game-data`. It owns source discovery, acquisition, schema validation, normalization, live sessions, domain publication, and provider health. `RelayBitCraftProvider` owns the HTTP-cache domains; typed runtime/session pairs commit their own normalized generations directly to `CurrentStateRepository`. The rest of the application should depend on their shared normalized contracts rather than on Relay DTOs.

3.3 Startup flow

At a high level:

- 1 `main.tsx` mounts the application, lazy-loads `AppShell`, and performs an initial auth request used for Firebase identity.
- 2 `AppShell` loads settings, auth/access state, legal state, page route state, release information, and display preferences.
- 3 The server initializes SQLite pragmas, schema/bootstrap migrations, prepared statements, sessions, providers, collectors, and delivery coordinators appropriate to its process role.
- 4 The provider discovers current Relay topology rather than hard-coding a database endpoint.
- 5 Durable last-good domain envelopes can be served immediately.
- 6 Relay HTTP and typed sessions reconcile newer current state in the background.
- 7 The selected page requests its owned domains and any specialized page endpoints.

One startup inefficiency is that the provider performs an initial claim/member/citizen HTTP fetch and the first scheduled reconciliation fetches all six HTTP domains, repeating some work. This is low priority compared with route, history, and market improvements.

3.4 Stable and deep modules

The strongest existing modules have narrow interfaces hiding substantial internal complexity:

- `GameDataProvider` in `src/server/game-data/contracts.ts` hides the acquisition provider.
- `RelayBitCraftProvider` in `relayProvider.ts` implements the Relay HTTP-cache acquisition adapter.
- `CurrentStateRepository` in `currentStateRepository.ts` hides staging, current envelopes, generation events, and durable last-good storage.
- `PageRefreshController` in `src/refresh/pageRefresh.mjs` hides coalescing, visibility, manual refresh, retry, and near-live policy.
- `Dialog` in `src/components/main/Dialog.tsx` hides portal, focus, Escape, stack, and scroll behavior.
- Specialized market and map modules hide complex projections and streaming from `AppShell`.

`AppShell` is less deep: its public role is simple, but it coordinates routing, authentication, access, settings, refresh, notifications, overlays, release state, and page composition in one large hook surface. It is a future maintainability seam, not a reason for a broad rewrite.

4. Relay data acquisition and publication

4.1 Topology discovery

`apps/bitcraft-local/src/server/game-data/topology.ts` discovers ready Relay databases and source metadata from health and cache-readiness responses. The application does not assume that a regional database name or endpoint is permanent.

The topology result identifies:

- the global database,
- regional databases,
- readiness,
- schema fingerprints where available,
- and the discoverable global/regional Relay source identities.

The server/runtime configuration, not topology discovery, selects the monitored claim and active-region scope. `relay-cache` is provenance assigned by the HTTP provider rather than a discovered topology source key.

Repeated topology failures are handled as operational health, not as a reason to clear last-good data.

4.2 Relay HTTP path

The HTTP adapter in `src/server/game-data/http.ts` applies:

- bounded request timeouts,
- one transient retry,
- a short circuit-breaker cooldown,
- structured errors,
- and source metadata.

Relay HTTP is used for already-proven, claim-oriented joined views such as:

- claim summary,
- members and citizens,
- inventories,
- crafts,
- deposits,
- and related settlement views.

The worker-owned provider normally reconciles the six HTTP-backed domains every 15 seconds, configurable and clamped between 5 and 60 seconds. Ordinary browser refreshes usually reread the local SQLite repository rather than performing Relay acquisition. A manual refresh can ask the provider coordinator to acquire upstream state, subject to cooling and single-flight behavior.

HTTP cache payloads do not use the typed binding fingerprint contract. That is not automatically wrong: they are a separate source contract. However, they currently lack an equally explicit version/fingerprint compatibility contract, so silent HTTP shape drift is a documentation and resilience gap.

4.3 Typed subscription path

Official generated TypeScript bindings under:

- `src/server/game-data/bindings/global`,
- `src/server/game-data/bindings/regional`,
- and `src/server/game-data/bindings/schema-manifest.json`

drive exact SpacetimeDB subscriptions. The project correctly avoids implementing its own wire codec.

The main typed runtimes include:

- `globalCatalogRuntime.ts` and `globalCatalogSession.ts`,
- `primaryRegionRuntime.ts` and `primaryRegionPlayerSession.ts`,
- `claimMarketRuntime.ts` and `claimMarketRegionSession.ts`,
- `regionalMarketRuntime.ts` and `regionalMarketRegionSession.ts`,
- `publicCraftRuntime.ts` and `publicCraftRegionSession.ts`,
- `empireRuntime.ts` and `empireRegionSession.ts`,
- `regionClaimsRuntime.ts` and `regionClaimsSession.ts`,
- map resource, spatial, and terrain runtimes and sessions.

The global catalog supplies item, cargo, recipe, building, skill, resource, equipment, buff, and region reference data. Regional subscriptions supply live operational rows such as orders, crafts, construction, research, recruitment, player state, claims, empires, terrain, and map entities.

Large cross-region features do not keep every region permanently connected. They use bounded region-session pools with connection caps, staged applies, idle close, and backoff.

4.4 Schema gate, review-time mismatch, and implemented recovery

Generated bindings are tied to exact schemas. The expected manifest captured on 17 August 2026 records:

- global expected fingerprint: `5814c18474097f92cd37c577a9c7f033c820a6d2dd7e679db936ba018a396f8c`,
- regional expected fingerprint: `3d0b4c9bba59f7b1daad5122369599ea557e333124c4f778079a45af1683f65b`.

The local smoke logs captured on 21 August 2026 observed a different global fingerprint:

- global observed fingerprint: `d8351a1bf26bc73cc9b589b2c7631a9d9042e626e51b8e1bbbc8c1ea74b91ee4`.

The runtime therefore stopped the affected generation and preserved last-good data. This was correct fail-closed behavior: changed wire shapes were not guessed into normalized data. The operational consequence in that dated smoke environment was that global catalog updates and dependent enrichments could not recover until a fresh schema was captured and validated. The dated value above is evidence from the original review, not a permanent live invariant.

Task 1 completed that recovery path. Fresh exact v9 response-byte hashes were captured as global `f538457640d0bb0d0dfa85577c4dfd71760854ba0fa9bb681c7b5de0e6e7f4db` and unchanged regional `3d0b4c9bba59f7b1daad5122369599ea557e333124c4f778079a45af1683f65b`. The repaired staged global generator output matched the tracked bindings byte-for-byte, so only the manifest/evidence changed. Global and regional live verification passed. A built six-domain smoke returned skills, buildings, research, public crafts, market, and region with zero warnings/partial errors; buildings required adding the already normalized 1,089-row catalogue to the provider-neutral publication. The fail-closed gate and last-good behavior were not weakened.

Schema download/mismatch evidence is now persisted with expected/observed fingerprints and a distinct typed runtime state. Matching publisher evidence clears stale blockers, while HTTP provider freshness can no longer make a typed subscription appear connected.

4.5 Normalization

`src/server/game-data/normalizers.ts` and focused projection modules turn source rows into provider-neutral domain records. Normalization is the point where the application should:

- convert source-specific fields and null shapes,
- retain source and receive timestamps,
- preserve exact 64-bit IDs as decimal strings,
- preserve large amounts without JavaScript precision loss,
- keep item type 0 and cargo type 1 as distinct identities,
- join labels only when evidence is available,
- reject rows outside the configured claim or region,
- and record partial warnings rather than inventing semantics.

The normalized contracts support both `sourceObservedAt` and `receivedAt`. In the inspected HTTP and typed market paths, `sourceObservedAt` is currently null, so their displayed age is local receipt age. Do not describe that as upstream observation age until a source timestamp is actually carried through.

This is a strong architectural choice. A junior developer adding Relay-backed data should extend a normalizer/projection and a provider-neutral contract rather than importing a generated table type into a React page.

4.6 Domain generations and last-good state

`currentStateRepository.ts` stores normalized domain envelopes in generic current-state tables. Each domain publication is atomic:

- 1 validate and normalize a complete snapshot for that domain,
- 2 commit the payload and metadata,
- 3 make it current,
- 4 publish a local generation event,
- 5 keep the previous generation when acquisition or validation fails.

This prevents partially written rows inside one domain. A response containing several domains may still combine independently current last-good generations by design, because partial availability is preferred to a blank application. Task 5 made that contract explicit: the API returns per-domain generation, freshness, confidence, age, provenance, warnings, and exact enrichment dependencies plus `meta.coherence` and the participating known application generations. Unknown or differently generated dependencies fail closed as mixed. Coherence is limited to local application publications and does not claim that upstream sources observed the world simultaneously.

4.7 Current state versus durable side effects

Publishing current state before slow history or notification work is good for page latency. At review time, the claim-market path had this unsafe hand-off:

```
Commit current market snapshot
  |
  process crashes
  |
in-memory transition callback never runs
```

Task 6 closed that crash window. `claimMarketRuntime.ts` now derives a compact version-1 transition and commits it with the winning market generation through `commitGenerationWithTransition`. Stale/equal generations publish neither state nor a transition. A worker-owned leased dispatcher applies idempotent market history, authoritative trades, activity, and

durable Discord enqueue in one transaction, then acknowledges with the active lease token. Restart tests prove a committed edge survives process close/reopen and replay inserts each effect once. Network delivery remains outside both provider publication and transition dispatch.

The implemented shape is:

- 1 commit the current generation and deterministic transition payload in one SQLite transaction,
- 2 return the current generation immediately,
- 3 lease and dispatch the durable transition asynchronously in bounded batches,
- 4 append history/activity and enqueue Discord idempotently,
- 5 acknowledge the transition only after all required durable effects succeed.

4.8 Health and provenance

gameDataRoute.ts can return per-domain:

- freshness: live, fresh, stale, or unavailable,
- confidence: authoritative, joined, partial, or unknown,
- age,
- warnings,
- provider and source key,
- database and region,
- schema fingerprint,
- source time and receive time provenance.

Two qualifications remain:

- Receipt age cannot distinguish genuinely fresh upstream data from an old source result received recently. The contracts can carry both timestamps, but the inspected HTTP and typed market adapters currently persist sourceObservedAt as null. Carry the source-observed value whenever a source actually provides it.
- Typed subscription heartbeats are now persisted and projected independently from HTTP provider freshness; schema-blocked, connected, and disconnected states remain distinct.

The topology/runtime health path now retains a sanitized schema URL/source, expected and observed fingerprints, attempt time, status, and bounded error. Credentials, schema bodies, and arbitrary upstream response text are not exposed.

5. Local API and browser data flow

5.1 Provider-neutral game-data route

The main page contract is:

```
GET /api/local/game-data?claimId={claimId}&domains={commaSeparatedDomains}
```

server.mjs dispatches the route around lines 7534-7675. gameDataRoute.ts composes domain envelopes, checks claim scope, applies catalog enrichment, and returns partial errors. A request receives:

- HTTP 200 when at least one requested domain is available, including stale last-good data,
- HTTP 503 only when no requested domain has ever loaded,
- domain quality metadata,
- a combined data structure used by the current client.

React selects domains in src/api/pageDomains.ts. src/api/gameData.ts fetches the contract, and src/api/gameDataLoader.ts retains same-page data across refreshes and caches page navigation payloads.

5.2 Specialized APIs

Not every page should force its data into the generic route. Focused APIs exist where the query shape is substantially different:

- Market: overview, regions, catalog, order book, price history, deals, buy orders, stalls, deal watches.
- Map: region/catalog metadata, snapshots, SSE invalidation, binary resource partitions, terrain and road tiles.
- Craft planning: plan summary, item detail, management, audit, and catalog search.
- Player detail: selected member/profile data.
- Activity: recent combined history and debounced full-history search.
- Empires: overview, details, watchtowers, claim members, and active regions.
- Auth, privacy, legal, popups, settings, notifications, and administrator resources.

This is a sensible deep-module pattern: a focused endpoint can return a compact view instead of exposing a general database abstraction.

5.3 Browser refresh policy

`src/refresh/pageRefresh.mjs` defines three policies:

- **Interval:** most public operational pages. Default is 30 seconds, configurable between 15 and 300 seconds.
- **Near-live:** Craft Monitor. Generation invalidation is coalesced, with bounded backoff and a recovery poll.
- **Manual-only:** Craft Calculator and Sync.

The controller:

- allows one active cycle,
- aborts superseded requests,
- coalesces near-live invalidations,
- retains displayed data while refreshing,
- pauses normal refresh in a hidden tab,
- catches up on visibility,
- and retries Craft Monitor's near-live failures at 5, 10, 20, then 30 seconds. Ordinary interval pages wait for the next configured interval.

Task 8 generalized the generation watcher to every provider-neutral page. One watcher is scoped to the active claim and that page's owned domains. SSE is the prompt invalidation path; Craft Monitor uses a one-second recovery poll and other interval provider pages use 30 seconds. Hidden tabs do not poll and perform one visibility catch-up. Manual-only and non-provider pages create no watcher. Generation-triggered failures retain independent 5/10/20/30-second backoff, coalesce to one trailing cycle, and cannot displace queued manual work.

5.4 Navigation cache behavior

Task 2 made loader state and completions scope-safe by `claimId:panel`: only a matching cache entry can seed a new scope, same-scope refresh retains its data, and late completions cannot overwrite a newer scope. Task 9 replaced the unbounded Map with an access-order LRU capped at 8 entries, 4 MiB of conservatively estimated resident bytes, and a five-minute absolute TTL. Oversized or size-unknown results remain available to the active caller but are not cached. Confirmed claim changes clear only the old claim. A full interactive route-loop heap measurement remains open.

5.5 Unnecessary history request

At review time, `src/api/localHistoryInclude.ts` always included activity and ordinary pages requested history they did not render. Task 8 replaced that with exact ownership:

- Dashboard: `activity,market,dashboard;`

- Activity: activity;
- Settlement Market: market;
- all other panels: no history request and no history refresh enrollment.

On the smoke server:

- include=activity&activityLimit=60 returned about 31,958 bytes.

The original 31,958-byte observation remains useful as review-time evidence of the removed request shape, not a current per-page behavior claim.

5.6 Startup configuration and auth

Task 11 added one public `/api/local/bootstrap` contract for public config, auth/session summary, legal state, and runtime build metadata. `main.tsx` waits for a validated decimal claim ID before mounting `AppShell`, passes the Firebase identity from the same request, and does not start claim-scoped hooks from `DEFAULT_CLAIM_ID`. The legacy `config/legal/auth` endpoints remain for compatibility. Protected Admin and bot settings still require authenticated routes and are excluded from bootstrap.

6. Domain ownership and data completeness

6.1 Main domain map

Domain	Primary source	Important consumers
claim, members, citizens	Relay HTTP joined cache	Dashboard, Members, most settlement pages
players	Primary-region typed state	Dashboard, Members, Map, Region
skills	Global catalog plus member joins	Professions, Leaderboard
inventories	Relay HTTP joined inventory data	Inventory, Construction, Craft Planning
crafts, contributions	Relay HTTP craft state plus regional typed evidence	Craft Monitor, Dashboard, Members
construction, research, recruitment	Regional typed state enriched by global catalog	Construction, Research, Members
market	Claim-scoped regional typed orders and closures	Dashboard, Local Market
regional-market	Bounded regional typed market sessions	Global Market
region, region-claims	Global reference state and regional claim sessions	Region, Dashboard, Empires
empires, deposits	Regional/global typed state and Relay deposits	Empires, Map
public-crafts	Bounded regional typed sessions	Public Craft Finder
catalogs	Global typed subscriptions	Most enrichment, calculator, market, map
map-spatial, map-resources, terrain	Specialized typed runtimes and tile packs	Map
history and analytics	Locally observed SQLite events	Dashboard, Activity, market charts, notifications

6.2 Observed local domain condition

The 21 August smoke snapshot is diagnostic evidence, not a permanent product guarantee:

- claim, members, citizens, players, inventories, inventory banks, crafts, equipment, region claims, and deposits were available.
- skills and buildings were unavailable.
- region was unavailable in the general domain response.
- public crafts were partial with 1,092 warnings.
- research was partial with 67 warnings.

- market was partial with 88 warnings.
- construction had 4 warnings.
- recruitment, contributions, and empires each had at least one partial warning.
- the response contained 1,261 partial errors/warnings in total.

Some generic keys such as map-static, map-spatial, map-resources, and catalogs are also served through specialized on-demand routes, so their absence from one all-domain probe was not by itself proof that the feature was blank.

Post-review Task 1 evidence supersedes the mismatch as current state. The built server returned all six recovery targets—skills, buildings, research, public crafts, market, and region—with available authoritative data and no partial errors. Global and regional typed health were connected and live verifiers passed. Future drift still fails closed and preserves last-good data.

Warning volume itself needs improvement. Thousands of row-level warnings make health hard to scan and can inflate API payloads. Aggregate repeated warning classes by domain and retain a small sample plus counts.

6.3 Semantics that upstream data does not currently prove

The application correctly refuses to invent several values:

- Siege cancellation cannot be distinguished reliably from a removed or otherwise unknown row, so it is rendered as `removed_or_unknown`.
- A confirmed completed sale does not expose purchaser identity, so purchaser remains null/unavailable.
- Regional 30-day trade volume is not a complete upstream aggregate. The application can show only confirmed locally observed sales and must display `observedSince`.
- Disappearing market orders are not automatically treated as sales.
- Deposit unknown state is not treated as active or harvestable.
- Cross-region coordinates are not compared as if they share one geometry.

These are genuine data gaps rather than implementation omissions. Product copy, API fields, and junior developer expectations should preserve that distinction.

6.4 Additional data-quality investigations

Smoke logs repeatedly included SpacetimeDB client messages such as "Updating a row that was not present in the cache" for progressive actions, claim-local state, and inventory state. These may reflect subscription predicate/update semantics or SDK behavior; they are not proof of lost rows. Add counted diagnostics and a focused reconnect/update test before changing logic.

Game assets intentionally support text or same-origin fallback when no verified local icon exists. The build found 522 unavailable catalog identities. That is visible completeness debt, but it is less urgent than live catalog recovery and should be resolved through the existing permission/provenance process.

7. Frontend shell, navigation, and shared behavior

7.1 Entry and route model

`main.tsx` lazy-loads `AppShell` and supplies a bootstrap loading/error boundary. `AppShell` then selects among:

- `/terms`,
- `/privacy`,
- `/bot` or a bot-specific hostname,
- the main dashboard shell.

Public pages use query routing:

```
/?page=dashboard  
/?page=market&tab=deals  
/?page=map&mapView=fullscreen
```

src/navigation.ts defines the public page list and aliases. src/navigation/routeState.ts parses and canonicalizes query state. AppShell uses history.pushState and popstate instead of a routing library. The mapView=fullscreen state removes normal application chrome; it does not invoke the browser Fullscreen API. Admin uses page=admin with admin={tab} and optional config={section}; /bot uses section={botSection}.

This is simple and adequate, but every new route must update navigation, route parsing, page domains, access targets, AppShell lazy imports, and page composition. Boundary tests are important because there is no framework router to enforce consistency.

7.2 Shared chrome and overlays

AppShell owns:

- sidebar and mobile navigation,
- command palette,
- manual refresh state,
- stale/loading/error shell,
- notification drawer and toasts,
- settings, theme, accessibility, and analytics consent,
- auth, legal acceptance, Discord linking, and account deletion,
- first-run tour,
- release banner,
- help/legal/cookie dialogs,
- administrator action,
- operational popups.

Most modals use src/components/main/Dialog.tsx, which provides portals, focus placement and return, a focus trap, Escape, stacking, backdrop behavior, and body scroll locking.

AppPopupManager.tsx now renders operational messages through that shared Dialog primitive. Dismissible announcements receive the same focus trap, Escape, restoration, stacking, and scroll-lock behavior as other application dialogs; required operational messages explicitly disable dismissal. The popup manager loads on mount, refreshes when the document becomes visible, and uses a coalesced five-minute visible-tab refresh so an already-open session can receive new announcements without polling while hidden.

7.3 Access and auth

The server remains authoritative. Browser access state controls navigation and presentation, while protected routes enforce sessions, permissions, origin, and CSRF. Do not treat a hidden tab or disabled button as security.

The sidebar Admin action appears only after the AppShell-owned administrator session resolves as authenticated. A direct Admin route reuses that resolved session in AdminPanel rather than performing a second /api/local/admin/me request. The bot route performs one protected-session load, then gates every save-capable control on authenticated /api/local/admin/settings; signed-out or failed protected-settings states stay fail-closed. Legal and privacy routes remain publicly available.

8. Page-by-page developer handbook

8.1 Dashboard

Purpose. The operational landing page summarizes settlement status, membership, active crafts, construction, research, local market, recent activity, and craft-plan needs. Cards link into more detailed pages.

Data. It requests claim, members, citizens, players, construction, market, research, crafts, region, and region-claims through `/api/local/game-data`. It also requests the dashboard/activity/market projection from `/api/local/history` and separately loads `/api/local/craft-plan`.

Refresh. Normal interval refresh. Existing card data stays visible during refresh.

Main files.

- `src/pages/DashboardPage.tsx`
- `src/pages/dashboardView.ts`
- `src/styles/dashboard.css`
- `src/api/pageDomains.ts`
- `src/api/localHistory.ts`

Review note. The measured game-data payload was about 317 KB and local history about 103 KB. This page is the best first target for response-size and projection telemetry, then compact card-specific projections if measurement justifies them.

8.2 Leaderboard

Purpose. Compares settlement members across contributions, professions, activity, market, and online/session views.

Data. Shared claim/member/citizen/player/skills domains plus `/api/local/leaderboard`. Tab/filter state is local.

Tabs.

- Contribution ranks durable contribution totals.
- Professions compares skill totals and selected professions.
- Activity summarizes locally observed member events.
- Market compares confirmed locally observed sales and value.
- Online / Sessions derives presence and recorded session duration.

Refresh. Normal interval refresh.

Main files.

- `src/pages/LeaderboardPage.tsx`
- `src/styles/leaderboard.css`
- `src/server/craftContributionLeaderboard.mjs`
- related route and prepared statements in `server.mjs`

Review note. Market leaderboard queries should use explicit time windows or rollups rather than unbounded trade history.

8.3 Members

Purpose. Roster, online/presence context, equipment, active work, recruitment state, and focused member details.

Data. Claim, members, citizens, players, equipment, crafts, and recruitment domains. Selecting a member loads `/api/local/player-data` for detail.

Refresh. Normal interval refresh; selected detail requests are abortable.

Main files.

- `src/pages/MembersPage.tsx`
- `src/pages/membersView.ts`
- `src/pages/memberPresence.ts`
- `src/styles/members.css`
- `src/server/game-data/playerDataService.ts`
- `src/server/game-data/playerPresenceService.ts`

8.4 Professions

Purpose. Searchable and sortable profession levels, tiers, and member capability.

Data. Claim, members, citizens, players, and skills domains. Filtering and sorting are browser-derived.

Refresh. Normal interval refresh.

Main files.

- `src/pages/SkillsPage.tsx`
- `src/pages/professionCapability.ts`
- `src/styles/skills.css`

Implementation note. Skills were unavailable in the 21 August smoke, but Task 1's recovered six-domain smoke returned authoritative skills with zero warnings. Future schema drift remains fail-closed.

8.5 Craft Monitor

Purpose. Tracks active production, completion/progress evidence, and member contributions.

Data. Claim, members, citizens, players, crafts, and contributions domains. Selected player detail comes from `/api/local/player-data`.

Refresh. Near-live. It uses claim/domain-scoped generation SSE plus a one-second recovery poll. Other provider-neutral pages now use the same scoped watcher with a 30-second recovery poll.

Main files.

- `src/pages/ProductionPage.tsx`
- `src/styles/production.css`
- `src/refresh/generationWatcher.mjs`
- `src/server/game-data/craftProjection.ts`
- `src/server/game-data/craftContributionAttribution.ts`
- `src/server/productionLifecycle.mjs`

Review note. Contribution warnings and partial catalog status should be shown at the affected panel, not only as a shell-wide stale indicator.

8.6 Craft Planning

Purpose. Shows settlement needs, shortages, source routes, progress, buffers, stock locations, and detailed item plans. Authorized administrators can manage targets and overrides.

Data. `/api/local/craft-plan` and `/api/local/craft-plan/detail`, plus catalog search/detail. Management checks `/api/local/admin/me` and uses CSRF-protected admin craft-plan mutations and audit endpoints.

Refresh. Normal page cycle for public summaries; manager dialogs isolate their own requests and mutations.

Main files.

- `src/pages/CraftPlanningPage.tsx`

- `src/pages/CraftPlanManagerDialog.tsx`
- `src/pages/CraftPlanningRouteChooser.tsx`
- `src/pages/craftPlanningEffortView.ts`
- `src/pages/craftPlanningFishingView.ts`
- `src/pages/craftPlanningNeedDetails.ts`
- `src/pages/craftPlanningNeedsBoard.ts`
- `src/styles/craft-planning.css`
- `src/server/craftPlanning.mjs`
- `src/server/craftPlanSources.mjs`
- `src/server/craftPlanEffortProgress.mjs`

Review note. This feature is complex but already extracted from AppShell. Catalog health and typed item/cargo identity are critical to its correctness.

8.7 Inventory

Purpose. Displays monitored settlement storage, grouped items/cargo, quantities, banks, and selected item details.

Data. Claim, members, and inventories domains; `/api/local/catalog/item-detail` for focused descriptions.

Refresh. Normal interval refresh.

Main files.

- `src/pages/InventoryPage.tsx`
- `src/styles/inventory.css`
- `src/server/game-data/inventoryProjection.ts`
- `src/server/catalogRepository.mjs`

Review note. The measured game-data response was about 277 KB. Domain-level caching and compact projections could help, but should follow payload and interaction measurements.

8.8 Construction

Purpose. Shows projects, required and present materials, progress, and completion/filter state.

Data. Claim, members, inventories, and construction domains. It has no separate page endpoint.

Refresh. Normal interval refresh.

Main files.

- `src/pages/ConstructionPage.tsx`
- `src/styles/construction.css`
- `src/server/game-data/constructionProjection.ts`

8.9 Research

Purpose. Presents research status and progression, with local filtering and display derivation.

Data. Claim, members, and research domains.

Refresh. Normal interval refresh.

Main files.

- `src/pages/ResearchPage.tsx`
- `src/pages/researchView.ts`

- `src/styles/research.css`
- `src/server/game-data/researchProjection.ts`
- `src/server/game-data/researchTierPresets.ts`

Review note. The 21 August smoke recorded 67 research warnings. The post-recovery six-domain smoke returned authoritative research with zero warnings. The client now groups/caps warnings and exposes domain provenance.

8.10 Local Market

Purpose. Shows current monitored-claim buy/sell orders and locally observed analytics in Live and Analytics tabs.

Data. Claim, members, and market domains for current state; `/api/local/market/history` for price/trade history.

Tabs. Live renders the current monitored-claim order book. Analytics reads locally confirmed history and preserves `observedSince` rather than implying complete upstream history.

Refresh. Normal interval refresh.

Main files.

- `src/pages/SettlementMarketPage.tsx`
- `src/styles/market.css`
- `src/server/game-data/claimMarketRuntime.ts`
- `src/server/game-data/claimMarketRegionSession.ts`
- `src/server/marketTransitionDispatcher.mjs`
- `src/server/relayMarketTransitions.mjs`
- `src/server/currentMarketViews.mjs`
- `src/server/marketActivity.mjs`

Implementation note. Task 6 now commits a compact transition with the winning market generation and dispatches idempotent `history/activity/Discord` enqueue through leased restart-safe work. Current listings still publish independently of those side effects.

8.11 Global Market

Purpose. A cross-region market workspace with Overview, Browse/Search, Deals, Buy Orders, Deal Watch, and Stalls.

Data. `/api/local/regions/active` and focused `/api/local/market` routes for overview, catalog, regions, order books, price history, deals, buy orders, stalls, and watches. Auth state gates user-owned Deal Watch configuration.

Tabs.

- Overview aggregates regional liquidity, hubs, movers, and favorite quotes.
- Browse searches typed catalog identities, opens item order books, and loads price history.
- Deals derives cross-region opportunities from current sell orders and route context.
- Buy Orders focuses on current demand and highest bids.
- Deal Watch manages user-owned thresholds and shows durable alert history.
- Stalls shows active barter-stall markers and their current trade orders.

Refresh. Page/subview requests refresh with the normal interval. Tabs are query-addressable.

Main files.

- `src/pages/MarketPage.tsx`
- `src/pages/market/MarketOverview.tsx`
- `src/pages/market/globalMarket.ts`
- remaining `src/pages/market` modules

- `src/styles/market.css`
- `src/server/game-data/regionalMarketRuntime.ts`
- `src/server/game-data/regionalMarketRegionSession.ts`
- `src/server/regionalMarketViews.mjs`
- `scripts/benchmark-favorite-quotes.mjs`
- `src/server/liveDealWatch.mjs`
- `src/server/dealAlerts.mjs`

Implementation note. Task 7 replaced the 20-request `Promise.all` fan-out with one bounded typed-identity request. The server caches a compact quote index by committed generation and normalized region scope, capped at 8 entries and 2 MiB. The retained same-fixture benchmark preserved quote/metadata parity and reduced request count from 20 to 1; it is local projection evidence, not a production latency claim.

8.12 Region

Purpose. Gives regional settlement context, region status, players, and claims.

Data. Claim, members, players, region, and region-claims domains.

Refresh. Normal interval refresh.

Main files.

- `src/pages/RegionPage.tsx`
- `src/styles/region.css`
- `src/server/game-data/regionClaimsRuntime.ts`
- `src/server/game-data/regionClaimsSession.ts`
- `src/server/relayActiveRegions.mjs`

Review note. Region was unavailable in the original smoke while region-claims was available. Task 1's recovered six-domain smoke returned the region domain authoritative with zero warnings.

8.13 Empires

Purpose. Tracks empires, regional claims, watchtowers, sieges, deposits, membership evidence, and known Hexite coverage.

Data. Deposits from the generic game-data route plus `/api/local/regions/active`, `/api/local/empires`, `/api/local/empires/watchtowers`, `/api/local/empires/claim-members`, and `/api/local/empires/details`.

Tabs. Overview summarizes empires and known coverage; Watchtowers combines tower, siege, member, and aligned-claim detail; Deposits filters the Relay deposit projection. Focused dialogs load empire, siege, watchtower-member, and aligned-claim evidence.

Refresh. Normal interval refresh. Detail dialogs load focused data.

Main files.

- `src/pages/EmpiresPage.tsx`
- `src/pages/empires/*`
- `src/styles/empires.css`
- `src/server/game-data/empireRuntime.ts`
- `src/server/game-data/empireRegionSession.ts`
- `src/server/empireViews.mjs`
- `src/server/empireMembership.mjs`
- `src/server/empireHexite.mjs`

- `src/server/game-data/siegeNotifications.ts`

Review note. Preserve explicit unknown semantics. Missing siege rows do not prove cancellation, and partial regional coverage must remain visibly partial.

8.14 Map

Purpose. Interactive operational map for claims, settlement players, resources, terrain, biome, roads, watchtowers, and map tooling. It supports a dedicated map-view route state, `page=map` with `mapView=fullscreen`, that removes normal application chrome.

Data. Claim, members, and players from generic game data; `/api/local/map/catalog` and `/api/local/map/regions`; map snapshot and resource SSE; binary resource partitions; terrain status and tiles; biome and road tiles.

Refresh. The map has specialized snapshot/resource streams and reconnect messaging in addition to the normal shell refresh.

Main files.

- `src/pages/MapPage.tsx`
- `src/pages/map/NativeMap.tsx`
- remaining `src/pages/map` modules
- `src/styles/map.css`
- `src/server/game-data/mapResourceRuntime.ts`
- `src/server/game-data/mapSpatialScopeManager.ts`
- `src/server/game-data/terrainRuntime.ts`
- `src/server/mapSnapshot.mjs`
- `src/server/mapResourceBinaryRoute.mjs`
- `src/server/mapTiles.mjs`
- `src/server/roadTileStore.mjs`
- `src/server/terrainTileStore.mjs`

Review note. This is appropriately a specialized deep module. Do not force binary partitions or tile data through the generic game-data envelope.

8.15 Activity

Purpose. Settlement audit trail and operational event explorer.

Data. `/api/local/history` supplies up to 2,000 recent activity entries on this page. A debounced `/api/local/activity` request searches full history.

Refresh. Normal interval refresh; filters and compaction are browser-side.

Main files.

- `src/pages/ActivityPage.tsx`
- `src/pages/activity/*`
- `src/styles/activity.css`
- `src/server/productionActivity.mjs`
- `src/server/marketActivity.mjs`
- `src/server/notificationActivity.mjs`

Implementation note. Task 10 added disabled/report-only retention diagnostics and narrow daily rollups. `activity_events` is not approved for pruning: actor-detail reader parity, ownership, production baseline, verified backup/root, and seven-day evidence remain open.

8.16 Public Craft Finder

Purpose. Searches public crafting opportunities across configured regions and can transfer a chosen location into Map.

Data. Claim and public-crafts domains plus active-region context.

Refresh. Normal interval refresh.

Main files.

- `src/pages/PublicCraftFinderPage.tsx`
- `src/pages/publicCraftMath.ts`
- `src/styles/public-craft.css`
- `src/server/game-data/publicCraftRuntime.ts`
- `src/server/game-data/publicCraftRegionSession.ts`
- `src/server/game-data/publicCraftProjection.ts`

Review note. The original smoke domain had 1,092 warnings. The recovered six-domain smoke returned authoritative public crafts with zero warnings, and Task 5 now bounds/group warning presentation if partial data recurs.

8.17 Craft Calculator

Purpose. Searches the catalog and recursively expands recipes for a selected output quantity.

Data. `/api/local/catalog/search` and `/api/local/recipe-detail`. It does not load the generic page domains.

Refresh. Manual-only because results are query-driven rather than an operational live dashboard.

Main files.

- `src/pages/CraftCalculatorPage.tsx`
- `src/styles/craftcalc.css`
- `src/server/gameCatalog.mjs`
- `src/server/itemProbability.mjs`

Implementation note. The global catalog was recovered and live-verified in Task 1. Future schema mismatch still blocks affected generations rather than guessing data.

8.18 Sync

Purpose. Embeds the configured BitCraft Sync board with retry and open-in-new-tab fallbacks.

Data. A validated `https://bitcraftsync.app` iframe. It intentionally uses no Claim Monitor game-data API.

Refresh. Manual-only.

Main files.

- `src/pages/SyncPage.tsx`
- `src/styles/sync.css`

8.19 Admin console

Purpose. Authenticated operations for status, health, configuration, diagnostics, analytics, empire membership, database inspection, administrators, linked accounts, audit, backups, popups, and Discord settings.

Data. AppShell resolves the administrator session and passes it to AdminPanel, avoiding a duplicate `/api/local/admin/me` request. Entering a tab then loads only that tab's resources, including `/admin/status`, `/admin/jobs`, `/admin/settings`, `/admin/analytics`, `/admin/tables`, `/admin/users`, `/admin/user-accounts`, or `/admin/backups`. Mutations require permissions, same-origin checks, and CSRF. The console preserves unsaved configuration and displays shared success/error feedback.

Tabs.

- Operations: Status, Server Health, Configuration, and Diagnostics.
- Insights: Analytics, Empire Membership, and Database inspection/exports.
- Access: Administrators, Linked Accounts, and Audit history.
- Maintenance: Backups and retention maintenance.

Refresh. Each section owns appropriate polling or manual actions. Status polls scheduled-job state every 1.5 seconds only while jobs are running. Server Health has a separate 15/30/60-second auto-refresh control.

Main files.

- `src/components/admin/AdminPanel.tsx`
- Status: `src/components/admin/AdminStatusOverview.tsx` and `AdminPanel.tsx`
- Server Health: `src/components/admin/ServerHealthSection.tsx`
- Configuration: `src/components/admin/AdminConfigurationNav.tsx`, `src/components/admin/AdminPopupsSection.tsx`, and the focused inline sections in `src/components/admin/AdminPanel.tsx`
- Diagnostics: `src/components/admin/AdminPanel.tsx`
- Analytics and Audit: `src/components/admin/AdminAnalyticsSection.tsx`
- Empire Membership: `src/components/admin/AdminEmpireMembershipSection.tsx`
- Database and Backups: `src/components/admin/AdminDataSection.tsx`
- Administrators and Linked Accounts: `src/components/admin/AdminAccessSection.tsx`
- `src/styles/admin.css`
- `src/styles/server-health.css`
- `src/styles/discord-admin.css`
- `src/styles/admin-empire-membership.css`
- `src/server/adminPermissions.mjs`
- `src/server/adminRequestGuards.mjs`
- `src/server/serverHealth.mjs`
- `src/server/routePerformance.mjs`
- `src/server/operationalHistoryRetention.mjs`
- `server.mjs` administrator routes

Implementation note. Task 10 replaced the contribution-event full-table materialization with one fixed SQL COUNT/conditional-aggregate result. Retention controls remain deletion-disabled and report-only.

8.20 Bot Control

Purpose. Dedicated /bot dashboard for Discord setup, notifications, YouTube, channels, roles, moderation, content, tests, and diagnostics.

Data. The shell authenticates through `/api/local/admin/me`. Opening Bot Control loads `status/discovery` as needed; Commands loads `/api/local/admin/discord/custom-commands`. Configuration and actions use the corresponding CSRF-protected `/api/local/admin/discord/*` route.

Sections.

- Setup: token/application/guild setup plus channel routing.
- Automation: notification rules and YouTube monitoring.
- Roles and onboarding: Role Manager, Craft Watch, Colour Roles, and Role Panels.
- Moderation: Moderation actions, Safety Rules, and Member Records.
- Community content: Posts and Events, Commands, and Community Tools.

- Troubleshooting: Command Tests and Delivery Diagnostics.

Refresh. Bot-section components are React-lazy and rendered conditionally, so only the selected section's implementation is fetched and rendered.

Main files.

- `src/components/admin/AdminPanel.tsx` in botOnly mode
- `src/components/bot/botSectionState.ts`
- `src/components/bot/lazySections.tsx`
- Setup: `src/components/bot/DiscordSetupSection.tsx`
- Channels: `src/components/bot/DiscordChannelsSection.tsx`
- Notifications: `src/components/bot/DiscordNotificationsSection.tsx`
- YouTube Monitor: `src/components/bot/DiscordYouTubeMonitorSection.tsx`
- Role Manager: `src/components/bot/DiscordRoleManagerSection.tsx`
- Craft Watch: `src/components/bot/DiscordCraftWatchRolesSection.tsx`
- Colour Roles: `src/components/bot/DiscordColourRolesSection.tsx`
- Role Panels: `src/components/bot/DiscordRolePanelsSection.tsx`
- Moderation: `src/components/bot/DiscordModerationSection.tsx`
- Safety Rules: `src/components/bot/DiscordSafetySection.tsx`
- Member Records: `src/components/bot/DiscordMemberRecordsSection.tsx`
- Posts & Events, Commands, and Community Tools: `src/components/admin/AdminPanel.tsx`
- Command Tests: `src/components/bot/DiscordTestsPanel.tsx`
- Delivery Diagnostics: `src/components/bot/DiscordDiagnosticsPanel.tsx`
- `src/styles/bot-dashboard.css`
- `src/styles/discord-admin.css`
- `src/server/discordSettings.mjs`
- `src/server/discordNotifications.mjs`
- `src/server/discordDeliveryMode.mjs`
- `src/server/discordOutboxLease.mjs`
- `src/server/youtubeMonitor.mjs`

Implementation note. Task 4 added renewable atomic SQLite outbox leases, token-conditioned completion, expiry recovery, and aggregate duplicate-risk diagnostics. Tests used record mode or loopback fake Discord only. Semantics remain at-least-once, not exactly-once.

8.21 Terms, Privacy, and legal dialogs

Purpose. `/terms` and `/privacy` are dedicated public legal pages. The normal shell also presents Help, Terms, Privacy/analytics consent, Discord sign-in, legal acceptance, and account-deletion flows.

Data. `/api/local/legal`, `/api/local/auth/me`, `/api/local/auth/legal/accept`, and `privacy/account` endpoints. Legal policy loads once per normal-shell mount. Privacy export is a download endpoint; account deletion begins Discord reauthentication at `/auth/privacy/reauth/start` and then performs a CSRF-protected DELETE `/auth/privacy/account` after confirmation.

Main files.

- `src/components/main/LegalDialogs.tsx`
- `src/legal/*`
- `src/components/main/UserSettingsDialog.tsx`

- `src/server/legalAcceptance.mjs`
- `src/server/legalPolicyDigest.mjs`
- `src/server/userPrivacy.mjs`
- `src/server/accountDeletion.mjs`
- `docs/privacy-operations-runbook.md`

8.22 Global overlays

Purpose. Command palette, notification drawer/toasts, user settings, Discord sign-in and legal-acceptance prompts, account deletion, help/terms/privacy/cookie dialogs, release updates, first-run tour, restricted-access states, and operational popups work across pages.

Data. Notifications use independent `/api/local/notification-activity` and `/api/local/market/deal-alerts` polling, so they continue to update while the user is on another page.

Main files.

- `src/AppShell.tsx`
- `src/api/bootstrap.ts`
- `src/components/main/Dialog.tsx`
- `src/components/main/AppPopupManager.tsx`
- `src/popups/appPopups.ts`
- `src/notifications/*`
- `src/popups/*`
- `src/tour/*`
- `src/styles/app-popups.css`

Implementation note. Task 11 moved operational popups onto the shared `Dialog`, including focus, Escape/dismissibility, restoration, stacking, and scroll locking, and added visible-tab/five-minute refresh coalescing. Mounted DOM behavior tests passed; no separate in-app browser click-through is claimed.

9. Backend responsibilities beyond Relay

`server.mjs` is large because it is both composition root and route surface. Supporting modules under `src/server` contain most focused behavior:

- **Authentication and authorization:** `passwordAuth.mjs`, `sessionLookups.mjs`, `adminPermissions.mjs`, `adminRequestGuards.mjs`, `httpCsrf.mjs`.
- **User/legal/privacy:** `userPrivacy.mjs`, `privacyRetention.mjs`, `privacyDeletionLedger.mjs`, `accountDeletion.mjs`, `legalAcceptance.mjs`.
- **Craft planning:** `craftPlanning.mjs` and the `craftPlan*` modules.
- **Market/history:** `regionalMarketViews.mjs`, `relayMarketTransitions.mjs`, `marketActivity.mjs`, `liveDealWatch.mjs`, `dealAlerts.mjs`.
- **Empires:** `empireViews.mjs`, `empireMembership.mjs`, `empireHexite.mjs`.
- **Map:** `mapSnapshot.mjs`, `binary/resource` pages, `tile stores/renderers`, `road` and `terrain` modules.
- **Discord/bot:** `discordSettings.mjs`, `discordNotifications.mjs`, `embeds`, `OAuth`, `delivery mode`, `command interactions`.
- **Operations:** `serverHealth.mjs`, `deploymentRuntime.mjs`, `scheduledJobs.mjs`, `jobBudget.mjs`, `backups/audit routes`.
- **Database:** `schemaBootstrap.mjs`, `schemaMigrations.mjs`, `preparedStatements.mjs`, `databasePragmas.mjs`.

A broad `server.mjs` rewrite would be risky. New work should deepen focused modules and leave `server.mjs` as route wiring. A useful extraction is one that turns a complex route block into a small request parser, a focused service interface, and a response mapper with behavioral tests.

10. SQLite ownership, history, and retention

10.1 Appropriate durable data

SQLite is the correct owner for:

- generic current last-good domain envelopes and health,
- restart-safe checkpoints and provider transitions,
- locally observed market, activity, membership, production, and notification history,
- outboxes and dedupe keys,
- user/admin sessions, permissions, settings, and linked accounts,
- privacy/legal/audit records,
- backup and deployment metadata.

It should not become a scheduled copy of every current Relay table merely to make pages convenient. Current data belongs to live generations; durable tables need an independent restart, history, query, audit, or user-state reason.

10.2 Growth risk

The privacy retention job prunes sessions, analytics, alert and delivery logs, audit rows, votes, and other personal/operational records. At the 21 August review, no comparable operational-history machinery existed for:

- `market_events`,
- `market_trades`,
- `activity_events`,
- and production contribution history.

Task 10 added report-only operational-history machinery, narrow daily rollups, watermarks/ingestion identity/mutation tracking, bounded dry-run diagnostics, machine-verifiable backup gates, and fail-closed report behavior after any pruned parity gap. Current runtime safety is intentionally stricter than the original rollout proposal:

- `operationalHistoryRetentionEnabled=false`;
- approved deletion allowlist [];
- no live prune HTTP route;
- scheduled and Admin paths are hard-coded to deletion-disabled dry-run;
- pruning code is bounded to 5,000 rows but unreachable for live tables under the empty allowlist;
- `production_contribution_events` is preview-only and excluded because repair requires its raw evidence.

The health contribution diagnostic now uses one fixed conditional-aggregate SQL row. Market daily readers can combine validated rollups and late raw rows, but complete reader parity is not proven for per-trade regional price detail or actor-detail activity/member reports. Consequently no current table is approved for pruning.

Review-time broad consumers included:

- market leaderboard can load every matching trade,
- regional observed trades can load up to 5,000 raw JSON rows,
- server health loaded the entire contribution event table every poll (fixed by Task 10).

Before any future production deletion:

- 1 identify each table's product owner plus report, repair, replay, and legal dependencies; production contribution events may be needed for attribution repair,
- 2 define product, reporting, repair, and legal requirements,
- 3 record oldest row, rows per day, table/index/WAL bytes, backup time, and query plans,
- 4 prove every long-range reader against the existing idempotent rollups or add only the narrow approved aggregate it requires,
- 5 expose deletion counts in Admin,
- 6 configure an approved production backup root and verify current hashes, temporary restore, and `PRAGMA integrity_check`,
- 7 enable bounded pruning in transactions separate from rollup construction,
- 8 collect seven consecutive dry-run days and test complete rollup/report equivalence before adding any table to the allowlist.

10.3 Database operational measurements

Add and retain:

- row counts and oldest/newest event time per history table,
- rows/day,
- main database and WAL size,
- checkpoint duration,
- backup duration and last verified backup,
- p95 query latency for history/search/leaderboard,
- retention preview and last delete count.

11. Security, privacy, and delivery review

11.1 Positive controls

- Discord bot tokens are not returned in public settings.
- GeolIP/license configuration is represented as configured/not configured.
- Admin mutations are authenticated and CSRF-protected.
- Preview Discord behavior is record-only by default.
- Personal-data retention and deletion paths are explicit and tested.
- Retired-table access is guarded rather than silently falling back.
- Browser pages receive only provider-neutral same-origin data.

11.2 Rate-limit identity

Task 3 now trusts `X-Forwarded-For` only when `req.socket.remoteAddress` is an explicit trusted peer (loopback defaults plus optional exact configured peers); otherwise it uses the socket address. The example Caddy configuration overwrites the header. Limiter state is partitioned and bounded so report-only churn cannot evict enforced security buckets.

Game-data, order-book, and favorite-quotes client profiles remain report-only. Independent global game-data and market projection gates enforce 8 active plus 16 queued requests and return bounded `503/Retry-After: 1` on saturation. The deployed Caddy configuration and public spoof behavior have not been verified, and neither required seven-day client-profile observation window has occurred; do not describe per-client limits as production enforcement.

11.3 Discord leasing

At review time the generic outbox selected pending/failed rows and sent before marking them sent. Task 4 implemented the safe protocol:

- 1 in one immediate transaction, select an eligible row and change it to sending with locked_at, locked_by, attempts, and lease expiry,
- 2 renew ownership before each bounded Discord request and send outside the database transaction,
- 3 mark sent only when the lease token still matches,
- 4 record retry/backoff on failure,
- 5 recover expired leases,
- 6 keep the canonical unknown-outcome handling for requests whose external acknowledgement cannot be proven.

Two real SQLite connections and two record-mode processes prove concurrent claim exclusion, expiry recovery, post-lock renewal timing, and stale-token rejection. This gives at-least-once external delivery, not exactly-once: a process can still stop after Discord accepts a request but before SQLite records sent. Duplicate-risk diagnostics expose that unavoidable acknowledgement window; canonical cutover unknown outcomes remain terminally suppressed. No real Discord destination was contacted.

12. Performance review

12.1 Measured local API shape

Representative smoke observations:

Request	Approximate JSON bytes	Observed local time
Dashboard game data	317,055	333 ms cold; 10-19 ms warm
Members game data	233,685	74 ms
Inventory game data	276,903	123 ms
Craft Monitor game data	118,325	32 ms
Dashboard history	102,914	159 ms
Unused 60-row activity history	31,958	3-11 ms warm
Market overview, region 19	41,453	77-534 ms
Active regions	852	144 ms

Production Caddy enables zstd and gzip, so network transfer is lower than the uncompressed figures. Node still reads, parses, projects, and serializes the full structures, and the browser still parses the decompressed JSON.

12.2 Hot paths

Public game data. `/api/local/game-data` still reads, enriches, and serializes potentially large domain payloads. It now has bounded duration, projection, response-byte, status, and would-limit telemetry plus a global 8-active/16-queued projection gate. Its client profile remains report-only.

Market order books and favorites. Ordinary order-book reads retain bounded telemetry/gating. Market Overview no longer sends 20 order-book requests: one `POST /api/local/market/favorite-quotes` handles up to 20 unique typed identities and indexes the scoped snapshot once per generation/scope.

Health polling. Contribution diagnostics now use one fixed SQL conditional-aggregate result rather than materializing the full event table.

History growth. Market, activity, and contribution history growth remains an operational concern. Report-only retention/rollup machinery exists, but live deletion is disabled, the allowlist is empty, and external approval/evidence gates remain open.

Browser polling. Provider-neutral pages now react to owned generation events with domain-scoped watchers, and only Dashboard, Activity, and Settlement Market request owned history. Normal intervals remain reconciliation guards.

Browser memory. Page payload caching remains page-scoped, but it is now bounded by 8 entries, 4 MiB, a five-minute absolute TTL, and LRU eviction.

12.3 Recommended performance approach

Do not begin with compression tweaks or a broad state-library rewrite. The highest-value order is:

- 1 Completed locally: bounded route projection/duration/byte/status telemetry and global capacity gates.
- 2 Completed: exact history ownership and zero-request behavior for unowned pages.
- 3 Completed: one typed favorite-quote batch with a generation/scope index.
- 4 Implemented but not activated: trusted-client profiles remain report-only until deployed proxy probes and seven-day evidence pass.
- 5 Completed: contribution-health reads use fixed SQL aggregates.
- 6 Implemented only as disabled/report-only: retention, rollups, backup gates, and fail-closed readers; production enablement remains open.
- 7 Completed: domain-scoped generation invalidation for provider-neutral pages.
- 8 Completed: bounded navigation cache; full browser heap evidence remains open.
- 9 Still measurement-driven: reduce additional page projections only where deployed evidence justifies it.

12.4 Candidate budgets after a baseline

These are candidate targets, not first-release gates. Capture production-like hardware, fixture size, concurrency, and normal client profiles first, then set accepted deltas:

- Warm provider-neutral current-state route p95 below 150 ms.
- Market favorite quotes p95 below 200 ms for 20 typed identities.
- One market Overview quote request per refresh, not 20.
- No single public JSON response above 500 KB without a documented specialized streaming/binary reason.
- Hidden tabs issue no normal page-domain or history refreshes; the independent release-health check is excluded unless it is intentionally changed.
- No history table has unbounded growth without an owner, retention rule, and rollout decision.
- Server Health query cost remains near-constant as event history grows.
- Event-loop p95 delay and heap remain stable during a 30-minute multi-tab market smoke.

13. UX, accessibility, and maintainability review

13.1 High-value UI fixes

- Completed: prevent previous route/claim data from entering a new page render.
- Completed: preserve and show bounded per-domain stale, partial, unavailable, provenance, and dependency status at the shell and affected panels.
- Completed: use the shared Dialog for operational popups.
- Completed: refresh operational popups on visible startup/visibility and a five-minute fallback without polling hidden tabs.
- Keep current data visible on same-scope refresh, which is already a good behavior.
- Completed: aggregate warning groups, cap groups/examples, and report omitted counts rather than rendering thousands of near-duplicates.

13.2 Test posture

The repository has extensive source-boundary and Node behavioral tests. That breadth caught many architectural regressions. Task 11 added mounted React DOM coverage for popup focus/trapping/Escape/restoration, Deal Watch auth invalidation, and protected bot settings. The weaker area remains broad interactive browser behavior:

- no broad rendered route-loop/throttled claim-switch test beyond the focused loader/AppShell state-machine coverage,
- no end-to-end test for per-domain stale status,
- focused request-count and cache-eviction tests now exist, but not a full browser heap/route-loop benchmark,
- no response-size or load budget test.

Add focused behavior tests before changing these paths. Avoid replacing the existing boundary suite; complement it with a small number of user-visible interaction tests.

13.3 AppShell direction

AppShell should not be split merely because it is large. Extract a cohesive module only when a task needs that seam. Good candidates are:

- a bootstrap/auth/config provider,
- a page-data coordinator,
- a notification/release scheduler,
- an overlay host.

Each extraction should reduce the number of state variables and effects AppShell must coordinate and expose a small explicit interface. Avoid introducing a new state framework without a measured problem.

14. Junior developer workflow

14.1 How to trace a page

For any page:

- 1 Find its query key in `src/navigation.ts`.
- 2 Find its lazy import and composition in `src/AppShell.tsx`.
- 3 Read its domain list in `src/api/pageDomains.ts`.
- 4 Inspect direct fetch calls in the page and its subcomponents.
- 5 Find the matching route in `server.mjs`.
- 6 Follow the route into a focused `src/server` module or `src/server/game-data` service.
- 7 If the data is Relay-backed, identify the runtime/session and normalizer/projection.
- 8 Check current-state versus history ownership.
- 9 Read the focused tests before modifying behavior.
- 10 Run the lightest required verification from `AGENTS.md`.

14.2 How to add a Relay-backed field

- 1 Prove the authoritative HTTP route or typed table and its semantics.
- 2 Add or update a fixture.
- 3 Update generated bindings only through the pinned schema procedure when needed.
- 4 Normalize the field in the provider layer.
- 5 Preserve exact IDs, amounts, timestamps, nulls, and typed item/cargo identity.

- 6 Add it to a provider-neutral projection or envelope.
- 7 Decide whether it is current-only or needs durable historical observation.
- 8 Expose it through the generic domain or a focused route.
- 9 Render freshness/partial state.
- 10 Test malformed, missing, stale, reconnect, and schema-mismatch behavior.

14.3 How to decide whether to add a table

Add durable storage when upstream history can expire, a transition or notification must survive restart, a user/admin/legal record owns the state, or an indexed projection has measured interactive value. Do not add a scheduled table simply to avoid reading a committed current generation.

14.4 Verification commands

From the repository root:

```
corepack pnpm install
corepack pnpm --filter @workspace/bitcraft-local run dev
corepack pnpm --filter @workspace/bitcraft-local run build
corepack pnpm --filter @workspace/bitcraft-local test
```

Built smoke:

```
node scripts/start-bitcraft-local-smoke.mjs --restart
curl.exe -s http://127.0.0.1:18449/api/local/health
```

Provider live verification:

```
corepack pnpm --filter @workspace/bitcraft-local run verify:relay-global-live
corepack pnpm --filter @workspace/bitcraft-local run verify:relay-region-live
```

Never use live Discord delivery during routine tests.

15. Ranked findings and proposed ownership

Review priority	Finding	Reconciled status	Primary owner/seam
P0 operational	Global schema mismatch blocked typed catalog reconciliation	Resolved and live-verified; future drift remains fail-closed	bindings, schemaManifest, globalCatalogRuntime
P1 correctness	Cross-scope route/claim data could briefly render	Implemented and behaviorally verified	gameDataLoader, AppShell
P1 durability	Market current commit lacked durable side-effect hand-off	Implemented with atomic transition plus leased dispatcher	claimMarketRuntime, currentStateRepository
P1 delivery	Generic Discord consumer lacked atomic lease	Implemented with renewable token leases; at-least-once caveat remains	Discord outbox consumer and schema
P1 performance	Market favorites created up to 20 order-book scans	Implemented as one bounded batch; local retained benchmark passed	MarketOverview, regionalMarketViews
P1 operations	Operational history had no explicit retention/rollup	Disabled/report-only machinery implemented; production enablement gates remain open	schema/history services, privacy/maintenance
P1 protection	Heavy public routes lacked bounded telemetry/gates	Global capacity gates active; client profiles report-only pending external gates	httpRateLimit, server routes, Caddy trust
P2 contract	Multi-domain responses did not declare coherence	Implemented with exact local generation/dependency semantics	currentStateRepository, gameDataRoute
P2 observability	Client discarded domain quality metadata	Implemented with bounded status/provenance/warnings	gameData.ts, loader, affected panels
P2 efficiency	All pages requested activity history	Implemented exact three-page history ownership	localHistoryInclude, localHistory
P2 refresh	Most pages did not use generation invalidation	Implemented for all provider-neutral pages	AppShell, generationWatcher, refresh controller
P2 health	Health scan grew with contribution history	Replaced by fixed SQL aggregates	server health query
P2 diagnostics	Typed/HTTP/schema health causes were conflated or lost	Implemented separate typed heartbeat and sanitized schema diagnostics	persistedRuntimeHealth, topology, Admin health
P2 accessibility	App popups did not implement shared modal behavior	Implemented through Dialog with mounted DOM coverage	AppPopupManager, Dialog
P3 startup	Auth/config bootstrap was duplicated; initial claim could be premature	Implemented one public bootstrap; provider HTTP startup duplication remains a lower-priority observation	main, AppShell, provider startup
P3 memory	Navigation cache had no TTL/size/claim invalidation	Implemented 8-entry/4 MiB/5-minute LRU bounds	gameDataLoader, pageNavigationCache
P3 maintainability	AppShell and server.mjs remain high-change-risk coordinators	Open; work stayed focused and no broad rewrite was attempted	incremental focused module extraction

16. Recommended roadmap

Implemented locally: restore truth and protect correctness

- 1 Persisted clear schema/typed-source diagnostics.
- 2 Captured/validated global and regional schemas with the pinned staged procedure and passed bounded live applies.
- 3 Fixed cross-scope browser state.
- 4 Added trusted-proxy identity and report-only client profiles; deployed proxy verification and observation remain open.
- 5 Added bounded route size/timing telemetry and retention diagnostics.
- 6 Implemented atomic renewable Discord leasing.

- 7 Added response generation/coherence/dependency metadata.
- 8 Moved market transitions to a durable leased post-commit dispatcher.

Implemented locally: remove measured waste

- 1 Batched favorite quotes and indexed current market orders by typed identity per generation/scope.
- 2 Stopped unused local-history requests.
- 3 Replaced contribution-health full-table reads with aggregates.
- 4 Preserved and rendered bounded per-domain quality.
- 5 Extended generation invalidation to provider-neutral pages.
- 6 Added disabled/report-only raw-history retention, rollups, dry-run reporting, backup gates, and fail-closed parity behavior; production enablement remains open.
- 7 Aggregated warning classes, samples, and omitted counts.

Remaining after evidence and external gates

- 1 Complete deployed proxy/spoof validation and both seven-day rate-profile windows before considering client enforcement.
- 2 Capture production-like performance, database, backup, heap, and event-loop baselines.
- 3 Complete retention reader parity, owner/dependency/legal approvals, verified production backup/root, seven dry-run days, and explicit deployment approval before any allowlist or deletion enablement.
- 4 Run the broad interactive route/claim and multi-tab heap/browser smokes that were unavailable locally.
- 5 Extract cohesive AppShell/server coordinators only as related work needs the seam; provider startup HTTP duplication remains a lower-priority candidate.
- 6 Tune supervisor intervals or page projections only when profiling proves value.
- 7 Complete missing verified asset coverage.

17. Source map

Start with these files:

- App entry and shell: `src/main.tsx`, `src/AppShell.tsx`.
- Navigation and route state: `src/navigation.ts`, `src/navigation/routeState.ts`.
- Page ownership: `src/api/pageDomains.ts`.
- Main loader: `src/api/gameData.ts`, `src/api/gameDataLoader.ts`.
- History loader: `src/api/localHistory.ts`, `src/api/localHistoryInclude.ts`.
- Refresh: `src/refresh/pageRefresh.mjs`, `src/refresh/generationWatcher.mjs`.
- Page implementations: `src/pages` and `src/pages/*` subdirectories.
- Shared components: `src/components/main`.
- Admin and bot: `src/components/admin`, `src/components/bot`.
- Server composition/routes: `server.mjs`.
- Worker entry: `worker.mjs`.
- Provider contracts: `src/server/game-data/contracts.ts`.
- Relay provider: `src/server/game-data/relayProvider.ts`.
- Topology and HTTP: `src/server/game-data/topology.ts`, `http.ts`.

- Normalization: `src/server/game-data/normalizers.ts` and focused projections.
- Current state: `src/server/game-data/currentStateRepository.ts`.
- Main API composition: `src/server/game-data/gameDataRoute.ts`.
- Schema/bindings: `src/server/game-data/bindings` and `schemaManifest.ts`.
- Database: `src/server/schemaBootstrap.mjs`, `schemaMigrations.mjs`, `preparedStatements.mjs`.
- Market: `src/server/game-data/claimMarketRuntime.ts`, `regionalMarketRuntime.ts`, `src/server/relayMarketTransitions.mjs`, `regionalMarketViews.mjs`.
- Map: `src/server/game-data/map` and *terrain*, `src/server/map*` and tile modules.
- Discord: `src/server/discord*` and `notification/outbox` sections of `server.mjs`.
- Tests: `apps/bitcraft-local/test`.
- Deployment: `DEPLOYMENT.md`, `deploy`, `scripts/start-bitcraft-local-smoke.mjs`.

18. Review limitations and next evidence

Before enforcing client limits or enabling retention, capture:

- endpoint p50, p95, p99, response bytes, status, and 429 rate,
- market snapshot row count and CPU time per projection,
- SQLite table/index/WAL bytes, rows/day, oldest row, and backup time,
- event-loop delay and heap during multi-tab market use,
- navigation-cache hit, miss, entry size, and eviction,
- partial/stale incidence and recovery time by domain,
- warning counts grouped by stable warning code,
- transition outbox lag and Discord lease recovery counts.

The source review and implemented task reports are comprehensive, and mounted DOM tests now cover the changed popup/auth surfaces. A production-like load run, deployed proxy probes, broad interactive visual/navigation pass, retention owner approvals, verified production backup/restore, and both seven-day rollout windows remain necessary before declaring the operational rollout complete. No production deployment, rate-profile enforcement, history deletion, or real Discord delivery is evidenced by this document.

19. Companion documents

- Implementation roadmap: `docs/superpowers/plans/2026-08-21-app-performance-reliability-improvements.md`.
- Detailed Relay/backend evidence: `docs/reviews/2026-08-21-relay-backend-source-audit.md`.
- Existing application overview: `docs/application-overview.md`.
- Existing developer guide: `docs/developer-guide.md`.
- Live-first policy and migration record: `docs/relay-migration/README.md`.
- Known upstream semantic limits: `docs/relay-migration/unresolved-semantics-2026-08-02.md`.
- Existing binding refresh procedure: `docs/superpowers/plans/2026-08-17-refresh-relay-bindings.md`.